

Estruturas de Dados - Unintese

<https://gemini.google.com/app/51d1e16439abf9e9>

User prompt: eu quero montar um cronograma de 8 aulas. Cada aula com 2:30h. elas serão ministradas on0line e síncronas

Response: Apresento a seguir uma proposta formal para a estruturação do seu cronograma de 8 aulas online e síncronas.

Considerando a carga horária de 2h30min por encontro, o planejamento abaixo é voltado para uma trilha completa de **Lógica Algorítmica e Python**, partindo da sintaxe básica até a Programação Orientada a Objetos (POO). O modelo também integra metodologias ativas, como a Aprendizagem Baseada em Problemas (PBL) e dinâmicas de gamificação para manter o engajamento no ambiente virtual.

Estrutura Padrão de Cada Aula (2h30min)

Para garantir o aproveitamento e evitar a fadiga no formato online síncrono, recomenda-se a seguinte divisão de tempo:

- **00h00 - 00h30 (30 min):** Abertura, revisão do encontro anterior e introdução teórica do novo conceito.
- **00h30 - 01h15 (45 min):** Demonstração prática (Live coding) com aplicação de raciocínio computacional.
- **01h15 - 01h30 (15 min):** Intervalo.
- **01h30 - 02h15 (45 min):** Atividade prática guiada (Resolução de desafios curtos/PBL).
- **02h15 - 02h30 (15 min):** Compartilhamento de soluções, feedback e encerramento.

Cronograma de Conteúdo: Da Sintaxe à POO

Aula 1: Fundamentos do Pensamento Computacional e Sintaxe Básica

- Introdução ao ambiente de desenvolvimento.
- Conceitos de variáveis, tipos de dados primitivos e operadores lógicos/matemáticos.
- *Prática:* Resolução de problemas matemáticos simples e entrada/saída de dados.

Aula 2: Estruturas de Controle e Decisão

- Controle de fluxo: condicionais (if, elif, else).
- Operadores relacionais e tabelas verdade na prática.
- *Prática:* Criação de um sistema de validação de regras de negócios simples.

Aula 3: Estruturas de Repetição (Laços)

- Loops (for e while).
- Controle de laços (break, continue).
- *Prática:* Desafios gamificados de repetição (ex: simulação de tentativas de acesso ou contadores de eventos).

Aula 4: Estruturas de Dados Fundamentais

- Trabalhando com coleções: Listas, Tuplas e Dicionários.
- Manipulação de índices, fatiamento (slicing) e iteração em coleções.
- *Prática:* Construção de um pequeno banco de dados em memória utilizando dicionários.

Aula 5: Modularização e Escopo

- Criação de Funções, passagem de parâmetros e retorno de valores.
- Entendimento de escopo global e local.
- *Prática:* Refatoração de códigos das aulas anteriores para utilização de funções reutilizáveis.

Aula 6: Introdução à Programação Orientada a Objetos (POO)

- Paradigmas de programação: transição do estruturado para POO.

- Classes, Objetos, Atributos e Métodos.
- *Prática*: Modelagem de entidades do mundo real (ex: sistema de cadastro de alunos ou clientes).

Aula 7: Aprofundamento em POO e Tratamento de Erros

- Pilares da POO: Encapsulamento, Herança e Polimorfismo.
- Tratamento de exceções (try, except, finally).
- *Prática*: Expansão do modelo anterior aplicando herança e garantindo que o programa não falhe com entradas incorretas.

Aula 8: Projeto Final (PBL - Problem-Based Learning)

- Integração de todos os conceitos abordados.
- Desenvolvimento de uma solução para um problema prático e real.
- *Prática*: Desenvolvimento síncrono assistido, seguido da apresentação das soluções pelos alunos.

Este escopo foi projetado para maximizar a retenção técnica e o engajamento dos alunos no ambiente online. Caso o foco dessas 8 aulas seja direcionado para outra tecnologia específica de sua atuação (como desenvolvimento mobile, Java, Kotlin ou um framework web como Django), ou caso deseje agendar essas sessões em um calendário com datas específicas, por favor, informe os detalhes para que o escopo seja devidamente ajustado.

User prompt: É um curso só sobre estrutura de dados. Você já montou os topicos na resposta anterior. Agora é separa-las em 8 aulas(sendo a ultima um projeto final)

Response: Compreendido. Peço desculpas pelo desvio. Ajustarei o escopo rigorosamente para o aprofundamento exclusivo nas Estruturas de Dados detalhadas na primeira resposta, distribuindo-as logicamente ao longo de 8 encontros.

A estrutura de tempo sugerida anteriormente (teoria, *live coding*, intervalo, prática e revisão) será mantida, pois é altamente eficaz para a retenção técnica em formato síncrono. O currículo abaixo integra a Aprendizagem Baseada em Problemas (PBL) em todas as aulas, garantindo que a teoria seja imediatamente testada em cenários do mundo real.

Cronograma: Estruturas de Dados e Lógica de Armazenamento

Aula 1: Fundamentos de Memória, Big-O e Arrays

- **Teoria**: Como a memória RAM funciona (alocação contígua). Introdução à notação Big-O para medir complexidade de tempo e espaço ($O(1)$, $O(n)$, $O(n^2)$).
- **Live Coding**: Criação, iteração e manipulação de Arrays/Vetores. Demonstração do custo computacional ao redimensionar arrays estáticos.
- **Prática (PBL)**: Desenvolvimento de um sistema de inventário simples com limite de capacidade, analisando o impacto de inserções no meio do vetor.

Aula 2: Listas Encadeadas (Linked Lists)

- **Teoria**: Superando as limitações dos Arrays. Conceito de nós (nodes) e ponteiros de memória. Diferenças entre listas simples e duplamente encadeadas.
- **Live Coding**: Implementação "do zero" de uma classe Node e uma classe LinkedList, programando as operações de inserção e remoção.
- **Prática (PBL)**: Construção de um sistema de navegação (botões "Próximo" e "Anterior" de um reprodutor de mídia), utilizando uma lista duplamente encadeada.

Aula 3: Pilhas (Stacks) e Filas (Queues)

- **Teoria**: Estruturas lineares com regras de acesso restritas: LIFO (Last In, First Out) para pilhas e FIFO (First In, First Out) para filas.
- **Live Coding**: Implementação de Stacks e Queues utilizando as Listas Encadeadas da aula anterior como base estrutural.
- **Prática (PBL)**: * *Pilha*: Criar um mecanismo de "Desfazer/Refazer" (Undo/Redo) para um editor de texto.
 - *Fila*: Simular um sistema de *matchmaking* para um jogo online, alocando jogadores na ordem de chegada.

Aula 4: Tabelas Hash (Dicionários/Mapas)

- **Teoria:** Acessando dados em tempo constante $O(1)$. O que são funções de hash, como ocorre o mapeamento de chaves para índices e o problema das colisões.
- **Live Coding:** Implementação de uma Tabela Hash do zero, demonstrando o tratamento de colisões por encadeamento (chaining).
- **Prática (PBL):** Criação de um sistema de cache ou um mecanismo de busca rápida para um catálogo de usuários, priorizando a velocidade de recuperação de dados via chave.

Aula 5: Árvores Binárias e Árvores de Busca (BST)

- **Teoria:** Transição para estruturas não lineares. Terminologia de árvores (raiz, folhas, altura). Propriedades da Árvore Binária de Busca (Binary Search Tree) para buscas em $O(\log n)$.
- **Live Coding:** Inserção e travessia em árvores (In-order, Pre-order, Post-order).
- **Prática (PBL):** Estruturação de um sistema hierárquico, como o sistema de diretórios de um computador ou a árvore de categorias de um e-commerce.

Aula 6: Grafos e Algoritmos de Travessia

- **Teoria:** Diferença entre Árvores e Grafos. Vértices e arestas. Representação por Matriz de Adjacência vs. Lista de Adjacência.
- **Live Coding:** Implementação de um Grafo não direcionado e demonstração da Busca em Largura (BFS) utilizando a estrutura de Fila (Queue) construída na Aula 3.
- **Prática (PBL):** Mapeamento de uma rede social restrita, identificando conexões ("amigos em comum") ou o caminho mais curto entre dois usuários.

Aula 7: Estruturas Avançadas (Heaps) e Preparação para o Projeto

- **Teoria:** Filas de Prioridade e o conceito de Heaps (Min-Heap / Max-Heap). Revisão geral de como escolher a estrutura correta para cada problema.
- **Live Coding:** Como um Heap mantém o maior (ou menor) elemento sempre acessível no topo com eficiência estrutural.
- **Prática (PBL):** Sistema de triagem de um hospital, onde pacientes são atendidos com base na gravidade do estado (prioridade) e não apenas pela ordem de chegada.
- *Briefing:* Apresentação do escopo do Projeto Final.

Aula 8: Projeto Final (PBL Integrado)

- **Dinâmica:** A aula é dedicada à construção de uma solução computacional completa e à defesa das escolhas arquiteturais.
- **O Desafio:** Os alunos recebem a especificação de um sistema complexo (ex: um simulador de rotas logísticas ou um sistema de gestão de estoque de alta performance).
- **Avaliação:** O foco não será apenas na sintaxe do código, mas em argumentar *por que* escolheram uma Tabela Hash em vez de uma Árvore de Busca para armazenar os dados, ou *por que* usaram um Grafo para modelar o problema principal.
- **Encerramento:** Apresentação síncrona das soluções, *code review* e *feedback* técnico.

User prompt: tem como dar todo esse curso com python? Se sim, mostre aula a aula com seus respectivos conteúdos e códigos para a parte do PBL

Response: Com toda certeza. Python é uma excelente escolha para ensinar Estruturas de Dados. Embora a linguagem possua abstrações de alto nível (como o tipo `list` que já funciona como um array dinâmico, ou `dict` que é uma tabela hash otimizada), é perfeitamente possível criar as estruturas "do zero" utilizando Programação Orientada a Objetos. Isso garante que a base técnica e a lógica computacional sejam compreendidas antes do uso das facilidades da linguagem.

Abaixo, detalho o cronograma ajustado com foco exclusivo em Python, incluindo as propostas de código base para a etapa de PBL de cada encontro.

Aula 1: Arrays e Gerenciamento de Memória

Conteúdo Teórico: Abstração de arrays estáticos em Python, notação Big-O para acesso $O(1)$ e inserção/remoção $O(n)$.
PBL: Desenvolvimento de um sistema de inventário restrito, simulando as limitações físicas de um array estático.

```
class Inventario:
    def __init__(self, capacidade):
        self.capacidade = capacidade
        self.tamanho_atual = 0
        # Simulando um array estático preenchido com None
        self.itens = [None] * capacidade

    def inserir(self, item):
        if self.tamanho_atual < self.capacidade:
            self.itens[self.tamanho_atual] = item
            self.tamanho_atual += 1
            print(f"Item '{item}' adicionado.")
        else:
            print("Erro: Inventário cheio (Array Index Out of Bounds).")

    def mostrar(self):
        print(f"Inventário: {self.itens}")

# Aplicação PBL
mochila = Inventario(3)
mochila.inserir("Poção de Vida")
mochila.inserir("Espada Longa")
mochila.inserir("Escudo")
mochila.inserir("Chave Mestra") # Disparará o erro simulado
mochila.mostrar()
```

Aula 2: Listas Encadeadas (Linked Lists)

Conteúdo Teórico: Superando a limitação de tamanho fixo. Criação de nós e manipulação de ponteiros de referência.
PBL: Construção do histórico de navegação de um reprodutor de mídia (Lista Duplamente Encadeada).

```
class No:
    def __init__(self, musica):
        self.musica = musica
        self.proximo = None
        self.anterior = None

class ReprodutorMidia:
    def __init__(self):
        self.atual = None

    def adicionar_musica(self, musica):
        novo_no = No(musica)
        if self.atual is None:
            self.atual = novo_no
        else:
            # Insere após a música atual para simplificar o exemplo
            novo_no.anterior = self.atual
            self.atual.proximo = novo_no
            self.atual = novo_no
        print(f"Tocando agora: {self.atual.musica}")

    def voltar(self):
        if self.atual and self.atual.anterior:
            self.atual = self.atual.anterior
            print(f"Voltando para: {self.atual.musica}")
        else:
            print("Não há música anterior.")

# Aplicação PBL
player = ReprodutorMidia()
player.adicionar_musica("Faixa 1 - Rock")
player.adicionar_musica("Faixa 2 - Jazz")
player.voltar()
```

Aula 3: Pilhas e Filas (Stacks & Queues)

Conteúdo Teórico: Regras de acesso LIFO (Pilha) e FIFO (Fila). Aplicações práticas no sistema operacional. **PBL:** Criação de um sistema de "Desfazer" (Undo) para um editor de texto utilizando Pilhas.

```
class EditorTexto:
    def __init__(self):
        self.texto_atual = ""
        self.historico = [] # Utilizando list do Python como Pilha

    def escrever(self, texto):
        self.historico.append(self.texto_atual) # Salva o estado atual na pilha
        self.texto_atual += texto
        print(f"Texto: '{self.texto_atual}'")

    def desfazer(self):
        if self.historico:
            self.texto_atual = self.historico.pop() # Remove o último estado (LIFO)
            print(f"Desfazer acionado. Texto: '{self.texto_atual}'")
        else:
            print("Nada para desfazer.")

# Aplicação PBL
editor = EditorTexto()
editor.escrever("Olá ")
editor.escrever("Mundo!")
editor.desfazer()
```

Aula 4: Tabelas Hash (Dicionários)

Conteúdo Teórico: Funções de Hash, tratamento de colisões via encadeamento (chaining) e busca em tempo médio $O(1)$. **PBL:** Implementação de um catálogo de busca sem usar o dict nativo do Python, lidando com colisões.

```
class TabelaHash:
    def __init__(self, tamanho):
        self.tamanho = tamanho
        # Lista de listas para tratar colisões por encadeamento
        self.tabela = [[] for _ in range(tamanho)]

    def _hash(self, chave):
        return sum(ord(char) for char in chave) % self.tamanho

    def inserir(self, chave, valor):
        indice = self._hash(chave)
        # Substitui se a chave já existir no bucket
        for item in self.tabela[indice]:
            if item[0] == chave:
                item[1] = valor
                return
        self.tabela[indice].append([chave, valor])

    def buscar(self, chave):
        indice = self._hash(chave)
        for item in self.tabela[indice]:
            if item[0] == chave:
                return item[1]
        return None

# Aplicação PBL
catalogo = TabelaHash(5)
catalogo.inserir("123", "João Silva")
catalogo.inserir("456", "Maria Souza")
print(f"Buscando 123: {catalogo.buscar('123')}")
```

Aula 5: Árvores Binárias de Busca (BST)

Conteúdo Teórico: Hierarquia e organização em $O(\log n)$. Como as regras de esquerda (menor) e direita (maior) aceleram a busca. **PBL:** Inserção e busca de IDs de produtos em um sistema de estoque.

```
class NoArvore:
    def __init__(self, valor):
        self.valor = valor
        self.esquerda = None
        self.direita = None

class ArvoreBusca:
    def __init__(self):
        self.raiz = None

    def inserir(self, valor):
        if not self.raiz:
            self.raiz = NoArvore(valor)
        else:
            self._inserir_recursivo(self.raiz, valor)

    def _inserir_recursivo(self, no_atual, valor):
        if valor < no_atual.valor:
            if no_atual.esquerda is None:
                no_atual.esquerda = NoArvore(valor)
            else:
                self._inserir_recursivo(no_atual.esquerda, valor)
        elif valor > no_atual.valor:
            if no_atual.direita is None:
                no_atual.direita = NoArvore(valor)
            else:
                self._inserir_recursivo(no_atual.direita, valor)

    def em_ordem(self, no_atual):
        if no_atual:
            self.em_ordem(no_atual.esquerda)
            print(no_atual.valor, end=' ')
            self.em_ordem(no_atual.direita)

# Aplicação PBL
arvore = ArvoreBusca()
for id_produto in [50, 30, 70, 20, 40]:
    arvore.inserir(id_produto)
print("Produtos ordenados na árvore:")
arvore.em_ordem(arvore.raiz) # Saída: 20 30 40 50 70
```

Aula 6: Grafos e Travessia

Conteúdo Teórico: Vértices, arestas e listas de adjacência. **PBL:** Mapeamento de uma rede social para encontrar conexões diretas.

```
class RedeSocial:
    def __init__(self):
        self.usuarios = {} # Lista de adjacência usando dicionário

    def adicionar_usuario(self, nome):
        if nome not in self.usuarios:
            self.usuarios[nome] = []

    def adicionar_amizade(self, u1, u2):
        if u1 in self.usuarios and u2 in self.usuarios:
            self.usuarios[u1].append(u2)
            self.usuarios[u2].append(u1) # Grafo não direcionado

    def mostrar_amigos(self, usuario):
        if usuario in self.usuarios:
            print(f"Amigos de {usuario}: {self.usuarios[usuario]}")

# Aplicação PBL
grafo = RedeSocial()
```

```

grafo.adicionar_usuario("Alice")
grafo.adicionar_usuario("Bruno")
grafo.adicionar_usuario("Carlos")

grafo.adicionar_amizade("Alice", "Bruno")
grafo.adicionar_amizade("Bruno", "Carlos")
grafo.mostrar_amigos("Bruno")

```

Aula 7: Filas de Prioridade (Heaps)

Conteúdo Teórico: Importação e uso da biblioteca padrão `heapq` do Python para manter estruturas onde a prioridade dita a ordem de saída. **PBL:** Sistema de triagem hospitalar por nível de gravidade.

```

import heapq

class TriagemHospital:
    def __init__(self):
        self.fila_espera = []
        self.contador = 0 # Para desempate em caso de mesma prioridade

    def adicionar_paciente(self, prioridade, nome):
        # heapq no Python é Min-Heap. Multiplicamos por -1 para virar Max-Heap
        heapq.heappush(self.fila_espera, (-prioridade, self.contador, nome))
        self.contador += 1
        print(f"Paciente {nome} triado com prioridade {prioridade}.")

    def atender_proximo(self):
        if self.fila_espera:
            prioridade, _, nome = heapq.heappop(self.fila_espera)
            print(f"Atendendo: {nome} (Prioridade {-prioridade})")
        else:
            print("Fila vazia.")

# Aplicação PBL
hospital = TriagemHospital()
hospital.adicionar_paciente(1, "Paciente Leve")
hospital.adicionar_paciente(5, "Paciente Crítico")
hospital.adicionar_paciente(3, "Paciente Moderado")
hospital.atender_proximo() # Atende o Crítico primeiro

```

Aula 8: Projeto Final (PBL Integrado)

O Desafio: Os alunos recebem um código inicial (esqueleto) para um **Simulador de Logística e Entregas**.

- Eles devem utilizar um **Grafo** para mapear os pontos de entrega.
- Uma **Tabela Hash** (ou Dicionário) para armazenar os detalhes dos pacotes.
- Uma **Fila de Prioridade** para determinar qual pacote entregar primeiro com base na distância ou urgência.

Código Base fornecido aos alunos:

```

class SistemaLogistica:
    def __init__(self):
        # TODO: Inicializar o Grafo de rotas
        # TODO: Inicializar a Tabela Hash de pacotes
        pass

    def cadastrar_rota(self, origem, destino, distancia):
        # TODO: Implementar adição de aresta no Grafo
        pass

    def registrar_pacote(self, id_pacote, destino, urgencia):
        # TODO: Salvar pacote na Tabela Hash e adicionar à Fila de Prioridade
        pass

    def processar_entregas(self):
        # TODO: Processar a fila retirando o pacote mais urgente primeiro

```

```
# TODO: Exibir o caminho percorrido no Grafo  
pass
```

```
# A partir deste template, os alunos desenvolvem a lógica síncrona,  
# integram as estruturas e apresentam no final da aula.
```

User prompt: vamos aula a aula Ministre pra mim a aula 1, pois preciso entender o conteúdo para passar a meus alunos]

Response: Para a condução desta sessão com suas turmas, é essencial que a abstração da linguagem Python seja temporariamente deixada de lado. Os alunos precisam compreender as restrições físicas da máquina antes de aproveitarem as facilidades da linguagem.

Abaixo está o detalhamento pedagógico e técnico para a ministração da Aula 1.

Aula 1: Fundamentos de Memória, Big-O e Arrays

1. Preparação Teórica (00h00 - 00h30)

O objetivo desta etapa é desmistificar o que acontece no hardware quando uma variável é criada.

A. Gerenciamento de Memória (Alocação Contígua) Explique aos alunos que a memória RAM é como um grande armário com milhares de gavetas numeradas (os endereços de memória). Quando criamos um **Array estático**, o computador procura um bloco de "gavetas" vizinhas (contíguas) que estejam vazias para guardar os dados.

- **Ponto de atenção para a turma:** Se declaramos um Array de tamanho 5, o sistema aloca exatamente 5 espaços contíguos. Se precisarmos de um 6º espaço posteriormente, não podemos simplesmente usar a próxima gaveta, pois ela pode estar ocupada por outro programa. O computador precisa procurar um novo bloco de 6 gavetas, copiar os 5 itens antigos e adicionar o novo. Essa é a base do custo computacional.

B. Notação Big-O Introduza a notação Big-O como uma métrica formal para avaliar o pior cenário de eficiência de um algoritmo.

- **Acesso em $O(1)$:** Encontrar um item pelo índice é instantâneo (Tempo Constante). O computador aplica uma fórmula matemática simples: `endereço_base + (índice * tamanho_do_dado)`. Ele não precisa percorrer a lista.
- **Inserção/Remoção em $O(n)$:** Inserir um elemento no início ou no meio do Array exige "empurrar" todos os elementos subsequentes uma casa para a direita para abrir espaço (Tempo Linear).

2. Demonstração Prática: Live Coding (00h30 - 01h15)

Neste momento, esclareça o paradigma do Python: a estrutura `list ([ ])` nativa do Python é, na verdade, um *Array Dinâmico* (implementado em C sob o capô). Ela mascara a complexidade de redimensionamento e deslocamento.

Para que eles aprendam a Estrutura de Dados real, você conduzirá a criação de um Array estático simulado.

```
class ArrayEstatico:  
    def __init__(self, capacidade):  
        # Define o tamanho máximo fixo na memória  
        self.capacidade = capacidade  
        self.tamanho_atual = 0  
        # Cria uma lista pré-alocada com valores nulos (simulando a RAM vazia)  
        self.dados = [None] * capacidade  
  
    def acessar(self, indice):  
        # Acesso em O(1)  
        if 0 <= indice < self.capacidade:  
            return self.dados[indice]  
        raise IndexError("Índice fora dos limites do Array.")  
  
    def inserir_no_final(self, valor):  
        # Operação em O(1) (quando há espaço)  
        if self.tamanho_atual < self.capacidade:  
            self.dados[self.tamanho_atual] = valor  
            self.tamanho_atual += 1  
        else:  
            raise OverflowError("Estouro de Memória: O Array atingiu sua capacidade máxima.")
```



```
def exibir(self):
    print(f"Estado da Memória: {self.dados}")
```

Dica de instrução: Execute o código e force o `OverflowError`. Os alunos precisam ver o erro acontecendo para entender a rigidez da estrutura.

3. Dinâmica PBL: Sistema de Inventário Restrito (01h30 - 02h15)

O Problema (Cenário a ser passado aos alunos):

"Vocês foram contratados para desenvolver o sistema de inventário de um jogo clássico de RPG. O personagem possui uma mochila de couro com espaços físicos limitados (exatamente 4 slots). Desenvolvam o sistema que permite ao jogador adicionar itens, visualizar a mochila e lidar com o erro de tentar guardar um item quando a mochila estiver cheia, impedindo que o jogo trave."

Objetivo Pedagógico: Aplicar a classe criada no *live coding* em uma regra de negócio real.

Resolução Esperada dos Alunos (Código de Referência):

```
class MochilaRPG:
    def __init__(self, slots):
        self.slots = slots
        self.itens_guardados = 0
        self.inventario = [None] * slots

    def coletar_item(self, item):
        print(f"Tentando coletar: {item}...")
        if self.itens_guardados < self.slots:
            self.inventario[self.itens_guardados] = item
            self.itens_guardados += 1
            print(f"[Sucesso] {item} guardado no slot {self.itens_guardados - 1}.")
        else:
            print(f"[Falha] A mochila está cheia! Você deve descartar algo para pegar {item}.")

    def abrir_mochila(self):
        print("\n--- Conteúdo da Mochila ---")
        for i in range(self.slots):
            status = self.inventario[i] if self.inventario[i] is not None else "[Vazio]"
            print(f"Slot {i}: {status}")
        print("-----\n")

# Simulação da execução pelos alunos
mochila_heroi = MochilaRPG(4)

mochila_heroi.coletar_item("Poção de Cura")
mochila_heroi.coletar_item("Espada de Ferro")
mochila_heroi.coletar_item("Mapa Antigo")
mochila_heroi.coletar_item("Tocha")

mochila_heroi.abrir_mochila()

# Testando o limite de memória (estouro da capacidade)
mochila_heroi.coletar_item("Amuleto Mágico")
```

4. Condução do Encerramento (02h15 - 02h30)

- **Revisão de Código:** Peça que 1 ou 2 alunos compartilhem a tela e mostrem como implementaram.
- **Fechamento Teórico:** Faça a pergunta central para a turma refletir: *"Se a nossa mochila precisasse aumentar de tamanho durante o jogo ao comprar um 'upgrade', como faríamos usando Array Estático?"*
- **Gancho para a Aula 2:** A resposta à pergunta acima (criar uma nova mochila maior e transferir item por item gastando tempo linear em $O(n)$) é o que motivará o aprendizado de **Listas Encadeadas** no próximo encontro, onde os alunos aprenderão a alocar memória dinamicamente sob demanda.

User prompt: agora faça o mesmo para a aula 2

Response: Avançando no cronograma, a segunda aula é o momento de romper com o paradigma da memória contígua e introduzir o conceito de alocação dinâmica estruturada através de ponteiros.

Abaixo apresento o planejamento pedagógico e técnico para a Aula 2.

Aula 2: Listas Encadeadas (Linked Lists) e Alocação Dinâmica

1. Preparação Teórica (00h00 - 00h30)

Retome imediatamente a pergunta deixada no final da Aula 1: *"Como aumentar a mochila sem precisar de um novo bloco contínuo de memória?"*

A. Quebrando a Contiguidade (Nós e Ponteiros) Explique que uma Lista Encadeada não precisa de "gavetas" vizinhas na memória RAM. Os dados podem estar espalhados em qualquer espaço livre. Para que eles não se percam, introduza o conceito de **Nó (Node)**. Cada Nó armazena duas coisas:

1. A informação útil (o dado).
2. O endereço de memória (ponteiro/referência) de onde está o próximo Nó.

B. Tipos de Encadeamento

- **Simplemente Encadeada (Singly Linked):** A viagem é de mão única. Cada nó só conhece quem vem depois dele.
- **Duplamente Encadeada (Doubly Linked):** A viagem tem mão dupla. Cada nó conhece quem vem antes e quem vem depois. Exige mais memória, mas facilita a navegação.

C. Análise Big-O (Trade-offs) Contraste com o Array da aula anterior:

- **Acesso:** Piora drasticamente. Para ler o 5º elemento, o computador precisa obrigatoriamente passar pelo 1º, 2º, 3º e 4º. É uma busca linear de tempo $O(n)$.
- **Inserção/Remoção:** Melhora substancialmente. Se já soubermos o local exato (como o início ou o fim), basta trocar para onde os ponteiros apontam. Não é necessário "empurrar" os outros elementos, resultando em uma operação de tempo constante $O(1)$.

2. Demonstração Prática: Live Coding (00h30 - 01h15)

O *live coding* desta sessão exige abstração por parte dos alunos. Construa a estrutura de uma Lista Simplemente Encadeada "do zero", passo a passo.

```
# Passo 1: Definir a unidade básica da estrutura
class No:
    def __init__(self, dado):
        self.dado = dado
        self.proximo = None # Inicialmente, não aponta para ninguém

# Passo 2: Construir o gerenciador da lista
class ListaEncadeada:
    def __init__(self):
        self.cabeca = None # O início da lista (Head)

    def inserir_no_inicio(self, dado):
        novo_no = No(dado)
        # O próximo do novo nó passa a ser a antiga cabeça da lista
        novo_no.proximo = self.cabeca
        # A cabeça da lista é atualizada para o novo nó
        self.cabeca = novo_no
        print(f"[{dado}] inserido no início.")

    def exibir_lista(self):
        atual = self.cabeca
        elementos = []
        while atual is not None:
            elementos.append(str(atual.dado))
            atual = atual.proximo
        # Mostra visualmente o encadeamento
        print(" -> ".join(elementos) + " -> None")

# Demonstração
```

```

lista = ListaEncadeada()
lista.inserir_no_inicio("Terceiro")
lista.inserir_no_inicio("Segundo")
lista.inserir_no_inicio("Primeiro")
lista.exibir_lista()

```

3. Dinâmica PBL: Reprodutor de Mídia (01h30 - 02h15)

O Problema (Cenário a ser passado aos alunos):

"Vocês estão desenvolvendo a lógica de um aplicativo de streaming de música. O usuário precisa de uma playlist (fila de reprodução) onde ele possa avançar para a próxima música ou voltar para a música anterior, sem limites pré-definidos de quantas faixas a playlist pode ter. Utilizem a lógica de uma **Lista Duplamente Encadeada** para resolver o problema dos botões 'Próximo' e 'Anterior'."

Objetivo Pedagógico: Evoluir o código visto no *live coding* (que era simples) para um modelo de duplo encadeamento (aplicação prática).

Resolução Esperada dos Alunos (Código de Referência):

```

class Faixa:
    def __init__(self, musica):
        self.musica = musica
        self.proximo = None
        self.anterior = None

class ReprodutorStreaming:
    def __init__(self):
        self.faixa_atual = None

    def adicionar_na_fila(self, musica):
        nova_faixa = Faixa(musica)
        if self.faixa_atual is None:
            self.faixa_atual = nova_faixa
            print(f"Playlist iniciada. Tocando agora: {self.faixa_atual.musica}")
        else:
            # Simulação simples: insere logo após a faixa atual
            nova_faixa.anterior = self.faixa_atual
            nova_faixa.proximo = self.faixa_atual.proximo

            if self.faixa_atual.proximo:
                self.faixa_atual.proximo.anterior = nova_faixa

            self.faixa_atual.proximo = nova_faixa
            print(f"Faixa [{musica}] adicionada como próxima.")

    def proxima_faixa(self):
        if self.faixa_atual and self.faixa_atual.proximo:
            self.faixa_atual = self.faixa_atual.proximo
            print(f">> Avançou para: {self.faixa_atual.musica}")
        else:
            print(">> Fim da playlist. Nenhuma música adiante.")

    def faixa_anterior(self):
        if self.faixa_atual and self.faixa_atual.anterior:
            self.faixa_atual = self.faixa_atual.anterior
            print(f"<< Voltou para: {self.faixa_atual.musica}")
        else:
            print("<< Início da playlist. Nenhuma música para trás.")

# Simulação da execução pelos alunos
player = ReprodutorStreaming()

player.adicionar_na_fila("Bohemian Rhapsody - Queen")
player.adicionar_na_fila("Stairway to Heaven - Led Zeppelin")
player.adicionar_na_fila("Hotel California - Eagles")

print("\n--- Ações do Usuário ---")
player.proxima_faixa()

```

```
player.proxima_faixa()
player.proxima_faixa() # Testando o limite final
player.faixa_anterior()
```

4. Condução do Encerramento (02h15 - 02h30)

- **Revisão de Código:** Peça que os alunos expliquem como implementaram as lógicas de verificação `if self.faixa_atual.proximo is not None` para evitar erros de `NoneType` (um erro muito comum ao lidar com ponteiros em linguagens tipadas, equivalente ao `NullPointerException`).
- **Fechamento Teórico:** Destaque o poder do duplo encadeamento para navegação livre, mas lembre o custo (memória extra gasta armazenando o ponteiro anterior).
- **Gancho para a Aula 3:** Questione os alunos: "E se precisarmos de uma lista estritamente controlada? Por exemplo, no botão de 'Desfazer' (Ctrl+Z) do nosso editor de código, onde o último comando digitado tem que ser obrigatoriamente o primeiro a ser desfeito, sem pular etapas?" Isso pavimenta a transição perfeita para **Pilhas (Stacks)** e **Filas (Queues)** na aula seguinte.

User prompt: proxima aula

Response: Chegamos a um ponto de virada no curso. Na Aula 3, os alunos deixarão de se preocupar tanto com "como a memória é alocada" (o foco das aulas 1 e 2) e passarão a focar nas **regras de negócio** de como os dados entram e saem.

Aqui está o planejamento estruturado para a **Aula 3: Pilhas (Stacks) e Filas (Queues)**.

Aula 3: Regras de Acesso com Pilhas e Filas

1. Preparação Teórica (00h00 - 00h30)

Resgate o gancho deixado na última aula: "Como criar uma estrutura estritamente controlada, onde não se pode inserir ou remover em qualquer lugar?"

A. Pilhas (Stacks) - O Princípio LIFO

- **Conceito:** *Last In, First Out* (O último a entrar é o primeiro a sair).
- **Analogia Visual:** Uma pilha de pratos de restaurante. Você só pode colocar um prato limpo no topo, e só pode retirar o prato do topo. Se tentar puxar o prato do meio, todos caem.
- **Uso Real:** O botão "Desfazer" (Ctrl+Z) do computador, o histórico de navegação ("Voltar") do navegador de internet, e a *Call Stack* (Pilha de chamadas de função) do próprio Python.

B. Filas (Queues) - O Princípio FIFO

- **Conceito:** *First In, First Out* (O primeiro a entrar é o primeiro a sair).
- **Analogia Visual:** Uma fila de banco ou de supermercado. Quem chega primeiro, é atendido primeiro. Entra-se pelo final da fila, sai-se pelo início.
- **Uso Real:** Filas de impressão (vários computadores mandando arquivos para uma mesma impressora), servidores web processando requisições de usuários, e *Matchmaking* de jogos online.

C. O Segredo do Python e a Complexidade (Big-O)

- Ambas devem operar em $O(1)$ para inserção e remoção, pois só manipulamos as extremidades. Não há "busca" no meio da estrutura.

2. Demonstração Prática: Live Coding (00h30 - 01h15)

O objetivo aqui é mostrar como implementar essas regras lógicas em Python.

O grande momento de reflexão para os alunos: Explique que usar uma lista comum do Python e fazer `lista.pop(0)` para remover o primeiro elemento de uma fila é um **desastre de performance**. Relembre a Aula 1: ao tirar o primeiro elemento do Array (lista do Python), todos os outros elementos precisam "dar um passo para a esquerda". Isso custa $O(n)$. Para resolver isso na fila, introduza a biblioteca nativa `collections.deque` (Double-Ended Queue), que é otimizada para inserções e remoções em $O(1)$ em ambas as pontas.

```

from collections import deque

print("--- Implementando uma PILHA (LIFO) ---")
pilha = [] # A lista nativa do Python é perfeita para pilhas
pilha.append("Página 1") # push
pilha.append("Página 2")
pilha.append("Página 3")
print(f"Pilha atual: {pilha}")

# pop() sem índice remove o último elemento em O(1)
removido_pilha = pilha.pop()
print(f"Desfez (removeu): {removido_pilha}")
print(f"Pilha após desfazer: {pilha}\n")

print("--- Implementando uma FILA (FIFO) ---")
# Usamos deque ao invés de list para garantir O(1) na remoção do início
fila = deque()
fila.append("Cliente A") # enqueue (entra no final)
fila.append("Cliente B")
fila.append("Cliente C")
print(f"Fila atual: {fila}")

# popleft() remove o primeiro elemento em O(1)
atendido = fila.popleft() # dequeue (sai do início)
print(f"Atendendo: {atendido}")
print(f"Fila após atendimento: {fila}")

```

3. Dinâmica PBL: Sistema de Call Center Integrado (01h30 - 02h15)

O Problema (Cenário a ser passado aos alunos):

"Vocês foram contratados para desenvolver o sistema de balcão de atendimento de um Call Center. O sistema precisa colocar os clientes em uma **Fila de Espera** e atendê-los em ordem. No entanto, o atendente costuma apertar o botão de 'encerrar chamado' sem querer. Portanto, vocês também precisam de uma **Pilha de Histórico** para permitir que o atendente desfça (recupere) a última ficha encerrada caso tenha cometido um erro."

Objetivo Pedagógico: Combinar as duas estruturas no mesmo sistema, forçando os alunos a usar o raciocínio correto para cada fluxo de dados.

Resolução Esperada dos Alunos (Código de Referência):

```

from collections import deque

class CallCenter:
    def __init__(self):
        self.fila_espera = deque() # FIFO: Para organizar quem chega
        self.historico_acoes = [] # LIFO: Para desfazer erros (Ctrl+Z)

    def novo_cliente(self, nome):
        self.fila_espera.append(nome)
        print(f"[Entrada] Cliente {nome} entrou na fila de espera.")

    def atender_proximo(self):
        if len(self.fila_espera) > 0:
            cliente_atendido = self.fila_espera.popleft() # O(1) - Remove do início

            # Salva a ação na pilha caso o atendente queira desfazer
            self.historico_acoes.append(cliente_atendido)
            print(f"[Atendimento] Atendendo cliente: {cliente_atendido}")
        else:
            print("[Atendimento] A fila de espera está vazia.")

    def desfazer_encerramento(self):
        if len(self.historico_acoes) > 0:
            # Retira o último atendido da pilha de histórico
            cliente_recuperado = self.historico_acoes.pop() # O(1) - Remove do fim

```

```

        # Devolve o cliente para o INÍCIO da fila de espera (já que foi um erro)
        self.fila_espera.appendleft(cliente_recuperado)
        print(f"[Desfazer] Erro! O cliente {cliente_recuperado} foi devolvido ao topo da fila.")
    else:
        print("[Desfazer] Não há ações recentes para desfazer.")

    def mostrar_painel(self):
        print(f"\nPainel - Fila atual: {list(self.fila_espera)}")
        print("-" * 30)

# Simulação da execução pelos alunos
sistema = CallCenter()

sistema.novo_cliente("João")
sistema.novo_cliente("Maria")
sistema.novo_cliente("Carlos")
sistema.mostrar_painel()

sistema.atender_proximo() # Atende João
sistema.atender_proximo() # Atende Maria
sistema.mostrar_painel()

sistema.desfazer_encerramento() # Ops, fechei a aba da Maria sem querer!
sistema.mostrar_painel()

sistema.atender_proximo() # Retoma o atendimento da Maria

```

4. Condução do Encerramento (02h15 - 02h30)

- **Revisão de Código:** Questione os alunos: *"Por que usamos `appendLeft()` na hora de desfazer um erro, em vez de `append()` normal?"* (A resposta deve ser: porque se foi um erro, o cliente recuperado não deve ir para o final da fila e esperar tudo de novo, ele deve voltar para a prioridade máxima no início da fila).
- **Fechamento Teórico:** Revise que Pilhas e Filas resolvem ordenação de processos incrivelmente bem em $O(1)$.
- **Gancho para a Aula 4:** Faça a provocação final: *"Nossa fila e nossa pilha estão ótimas. Mas e se o gerente do Call Center quiser pesquisar especificamente os dados do cliente 'Carlos' no nosso banco de dados gigante, sem ter que atender um por um na fila até achá-lo? Como acessar um dado específico instantaneamente pelo nome ou CPF?"* Essa reflexão é a ponte exata para introduzir **Tabelas Hash (Dicionários)** no encontro seguinte.

User prompt: Agora faça um convite para a aula 2 de estrutura de dados. O tema é listas encadeadas

Response: **Assunto:** Convite para a Aula 2 - Rompendo os Limites da Memória com Listas Encadeadas

Prezados alunos,

Espero que todos estejam bem.

É com grande satisfação que os convido para o nosso segundo encontro síncrono da trilha de Estruturas de Dados em Python.

Em nossa primeira aula, compreendemos o rigor da alocação de memória contígua e finalizamos com um desafio arquitetural fundamental: *como expandir um sistema, como o nosso inventário, quando não há mais blocos vizinhos disponíveis na memória RAM?*

Nesta sessão, daremos o próximo passo evolutivo na engenharia de software. Abandonaremos a rigidez dos Arrays estáticos e introduziremos o conceito de **Alocação Dinâmica** por meio das **Listas Encadeadas (Linked Lists)**.

Agenda do Encontro (Duração: 2h30min)

- **Fundamentos Teóricos:** A quebra da contiguidade. O conceito de Nós (*Nodes*) e a estrutura de ponteiros e referências de memória.
- **Live Coding:** Construção de uma Lista Simplesmente Encadeada "do zero", compreendendo o ganho de performance em inserções e remoções de dados ($O(1)$).
- **Prática Orientada (PBL):** Desenvolvimento do sistema de navegação de um reprodutor de mídia em streaming (lógica dos botões de avançar e retroceder faixas) utilizando uma Lista Duplamente Encadeada.

Informações de Acesso

- **Data:** [Inserir Data]
- **Horário:** [Inserir Horário de Início] às [Inserir Horário de Término]
- **Local:** [Inserir Link da Sala Virtual]

Recomendo que revisem os códigos da nossa última aula e garantam que o ambiente de desenvolvimento Python esteja devidamente configurado para a nossa atividade prática.

Aguardo a presença de todos para mais uma sessão de alto nível técnico.

Atenciosamente,

[Seu Nome/Assinatura]

Professor de Tecnologia da Informação

User prompt: Agora faça um convite para a aula Aula 3 de estrutura de dados. • Pilhas (Stacks) e Filas (Queues)

Response: **Assunto:** Convite para a Aula 3 - Controle Estrito de Fluxo com Pilhas e Filas

Prezados alunos,

Espero que todos estejam bem.

É com grande satisfação que os convido para o nosso terceiro encontro síncrono da trilha de Estruturas de Dados em Python.

Em nossa última aula, rompemos as barreiras físicas da memória RAM e ganhamos total liberdade de navegação utilizando as Listas Encadeadas. Agora, nossa jornada avança para o cenário oposto: como arquitetar um sistema quando as regras de negócio exigem um controle estrito e rigoroso sobre a ordem exata em que as informações entram e saem?

Nesta sessão, mergulharemos no funcionamento das **Pilhas (Stacks)** e **Filas (Queues)**. Compreenderemos como o seu computador gerencia o botão "Desfazer" (Ctrl+Z) e como grandes servidores organizam milhões de requisições de clientes sem travar.

Agenda do Encontro (Duração: 2h30min)

- **Fundamentos Teóricos:** A lógica por trás dos princípios LIFO (*Last In, First Out*) e FIFO (*First In, First Out*).
- **Live Coding:** A implementação otimizada em Python. Entenderemos por que usar listas comuns para filas pode ser um desastre de performance e como a biblioteca `collections.deque` resolve isso garantindo operações em tempo constante $O(1)$.
- **Prática Orientada (PBL):** Construção do motor lógico de um Call Center Integrado. Desenvolveremos um sistema que gerencia uma fila de espera para clientes e, simultaneamente, utiliza uma pilha de histórico para recuperar atendimentos encerrados por engano.

Informações de Acesso

- **Data:** [Inserir Data]
- **Horário:** [Inserir Horário de Início] às [Inserir Horário de Término]
- **Local:** [Inserir Link da Sala Virtual]

Recomendo que cheguem com o ambiente Python preparado, pois nosso foco será converter essa teoria em código eficiente logo na primeira hora de aula.

Aguardo a presença de todos para mais um excelente momento de desenvolvimento técnico.

Atenciosamente,

Prof. Bruno Campos

Professor de Tecnologia da Informação

User prompt: faça a mensagem mais curta e mais informal ela começa em instantes, às 19:30h.

Response: Fala, turma! Tudo bem?

Passando para avisar que a nossa Aula 3 de Estrutura de Dados começa em instantes, às **19:30h!**

Hoje o tema é **Pilhas (Stacks) e Filas (Queues)**. Vamos colocar a mão na massa para entender como o computador faz o famoso "Ctrl+Z" (LIFO) e como organizar filas de atendimento de forma otimizada (FIFO) sem travar o sistema.

Preparem o café, abram o Python e bora codar!

📌 **Acesso à aula:** [Inserir Link da Sala Virtual]

Até daqui a pouco!

Prof. Brunno Campos

User prompt: eu manei dessa mensagem no grupo na segunda, mas a aula só será hoje. eu ja me desculpei e expliquei no equivoco. Agora eu vou manda a mensagem, pois a aula será hjoje e eu quero adicionar no texto uma piada sobre esse ocorrido

Response: Fala, turma! Tudo bem?

Dessa vez é oficial (prometo que não é um alarme falso da máquina do tempo como na segunda-feira)! 🤪

Nossa Aula 3 de Estrutura de Dados começa em instantes, **HOJE às 19:30h.**

O tema é **Pilhas (Stacks) e Filas (Queues)**. Vamos entender como organizar fluxos de atendimento (FIFO) e como o computador faz o famoso "Ctrl+Z" (LIFO) — que, aliás, seria muito útil para eu ter cancelado aquela minha mensagem de segunda, né? 🤪

Preparem o café, abram o Python e bora codar!

📌 **Acesso à aula:** [Inserir Link da Sala Virtual]

Até daqui a pouco!

Prof. Brunno Campos
